

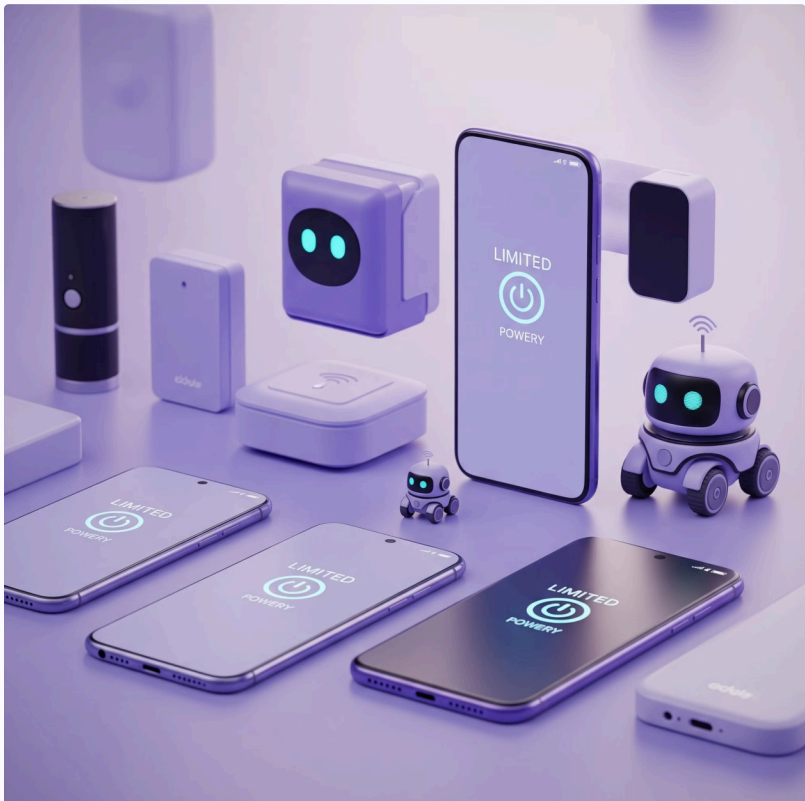


Pruning and Model Compression Basics

Slimming down AI models for edge deployment

A practical guide for ML engineers and applied researchers

Why Compression?



Edge devices = **limited compute, memory, and power**

Large AI models = slow inference, high latency

Compression reduces:

- Model size (disk/RAM footprint)
- Compute cost (fewer FLOPs)
- Power consumption

Goal: faster, leaner models without major accuracy loss

What Is Pruning?

Definition

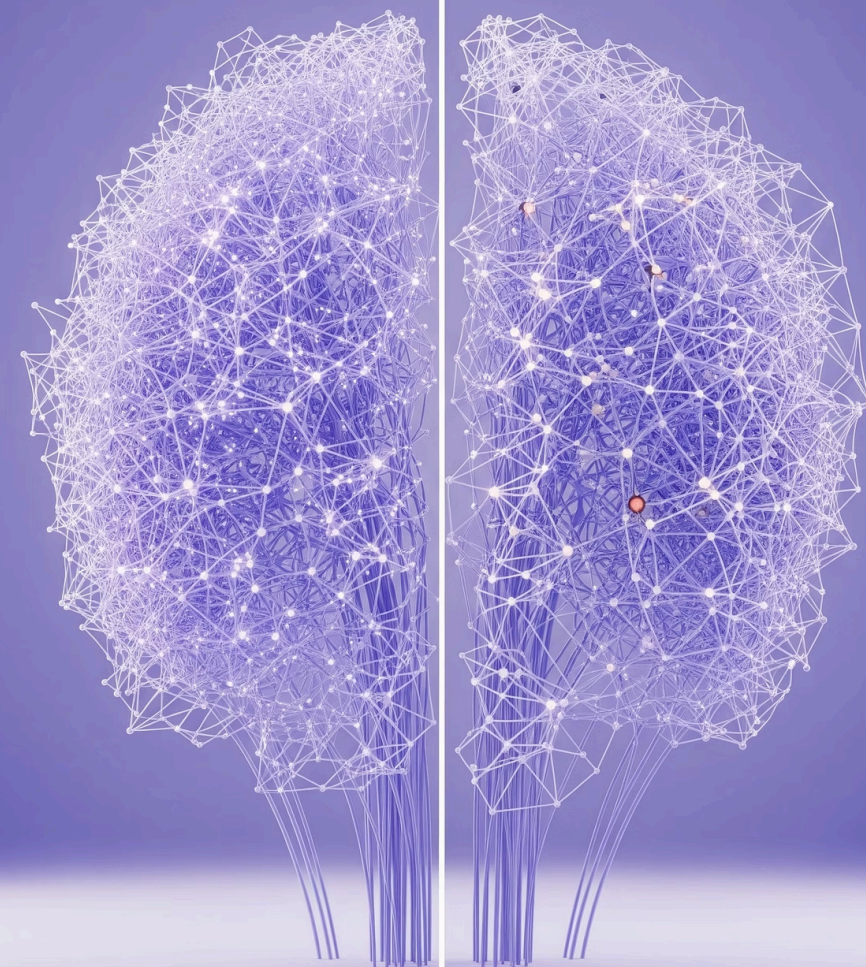
Removing **unnecessary weights, neurons, or filters** from a neural network

Based on

- **Magnitude** (drop small weights near zero)
- **Structured** (drop entire channels/layers)
- **Unstructured** (random connections removed)

Recovery

Pruned models retrain → recover performance

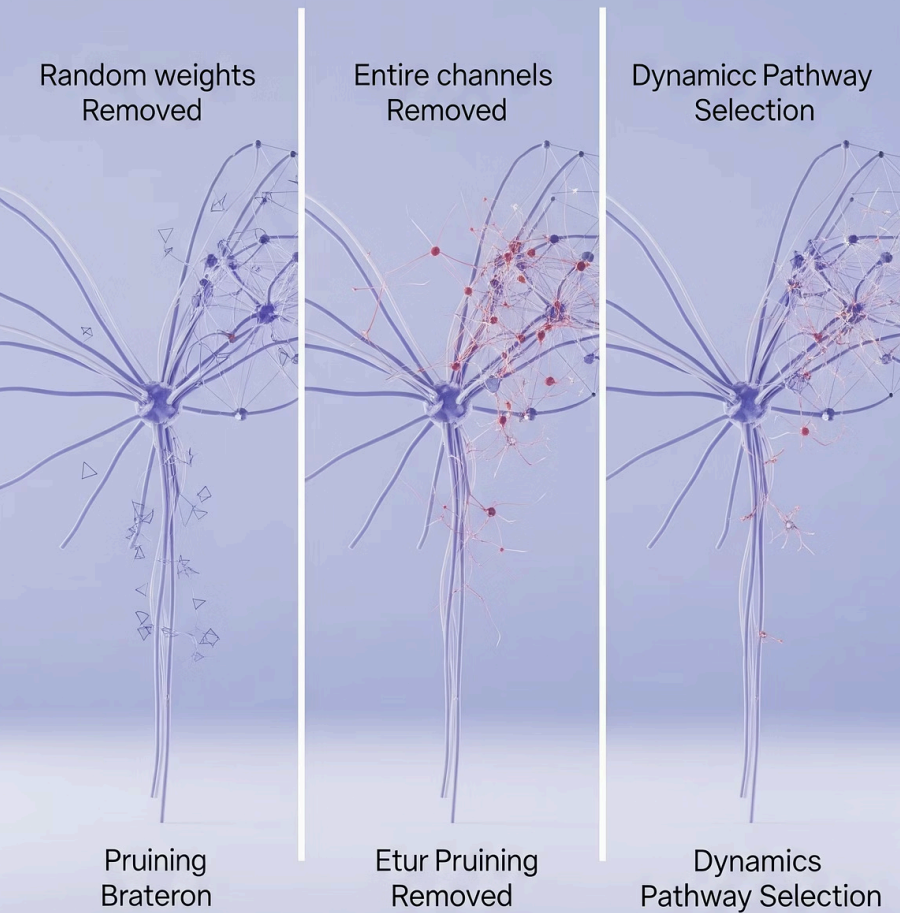


BEFORE

AFTER

Types of Pruning

Pruning Pattern



1

Unstructured Pruning

- Fine-grained (individual weights)
- Irregular sparsity → harder to accelerate on hardware

2

Structured Pruning

- Remove neurons, filters, heads, layers
- Regular → easier speedups on CPUs/GPUs

3

Dynamic Pruning

- Drop computations on the fly (adaptive execution)
- Context-dependent calculations

Pruning Workflow



Train baseline model

Establish performance benchmark



Apply pruning

Structured/unstructured based on target hardware



Fine-tune retrain

Regain accuracy lost during pruning



Export compressed model

Format for target platform



Deploy with sparsity-aware runtime

TensorRT, ONNX, PyTorch Sparse

Example: Unstructured Pruning (PyTorch)

```
import torch
import torch.nn.utils.prune as prune

# Load pre-trained model
model = torch.hub.load('pytorch/vision',
                       'resnet18',
                       pretrained=True)

# Prune 30% of connections in first conv layer
prune.l1_unstructured(model.conv1,
                     name="weight",
                     amount=0.3)

# Fine-tune model on training data after pruning
```

Result: Smaller parameter set, but accuracy drop requires retraining.

Implementation Notes

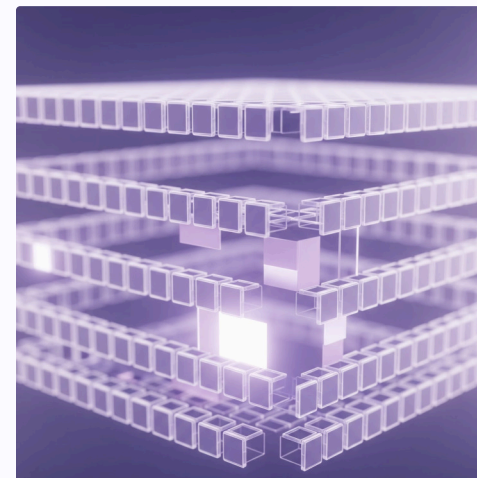
- PyTorch pruning is non-destructive (masks)
- Set parameters permanent with `remove()`
- Can be applied layer-by-layer or globally



Example: Structured Pruning (Filters)

```
# Remove entire filters from Conv2d layer
prune.ln_structured(
    model.layer1[0].conv1,
    name="weight",
    amount=0.2, # Remove 20% of filters
    n=2,       # L2 norm
    dim=0      # Filter dimension
)
```

Removes **20% of filters** → reduces FLOPs + speeds inference.



Hardware Benefits

Unlike unstructured pruning, removing entire filters creates a physically smaller network that directly translates to hardware speedups.

Compression Beyond Pruning



Quantization

Reduce precision (FP32 $\rightarrow$ INT8) to decrease memory footprint and computation time while maintaining accuracy through calibration.



Weight Sharing

Reuse weights through Huffman coding or clustering similar parameters to reduce unique parameter count.



Knowledge Distillation

Train small "student" model from large "teacher" model by mimicking output distributions rather than hard labels.



Low-rank Factorization

Approximate weight matrices as products of smaller matrices to reduce parameter count and computation.

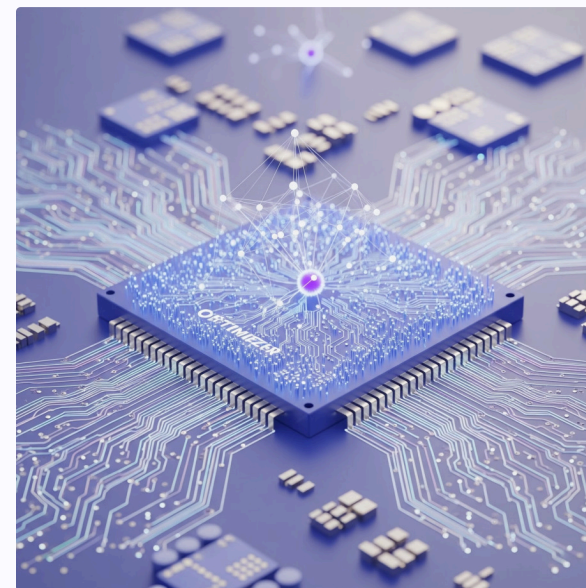
Hardware Acceleration of Sparse Models

Hardware Support Challenges

- GPUs/CPUs often ignore random sparsity → no speedup
- Memory access patterns matter more than FLOP count
- Structured pruning aligns with hardware → actual latency reduction

Acceleration Libraries

- NVIDIA TensorRT
- cuSparse
- ONNX Runtime
- TensorFlow Lite
- ARM Compute Library



⊗ Without sparsity-aware runtimes, pruned models may not deliver real-world speedups!

Trade-offs



Accuracy vs. Compression

Aggressive pruning = higher accuracy loss



Implementation Complexity

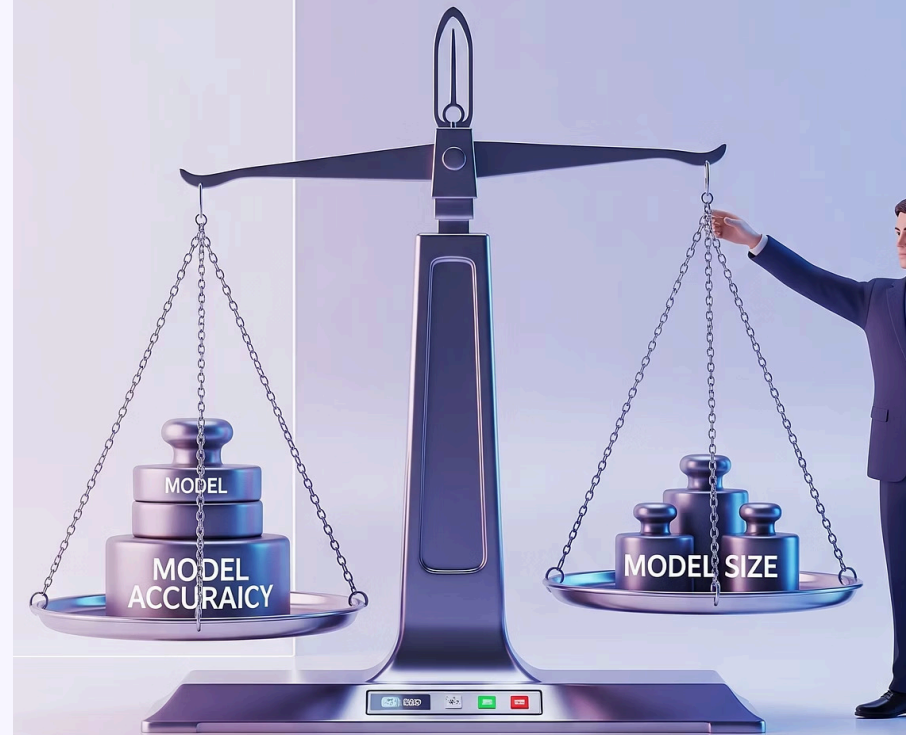
Sparse models harder to optimize without runtime support



Structure vs. Flexibility

Structured pruning = real speedups but less flexible

Need careful balance: compression vs accuracy vs hardware support

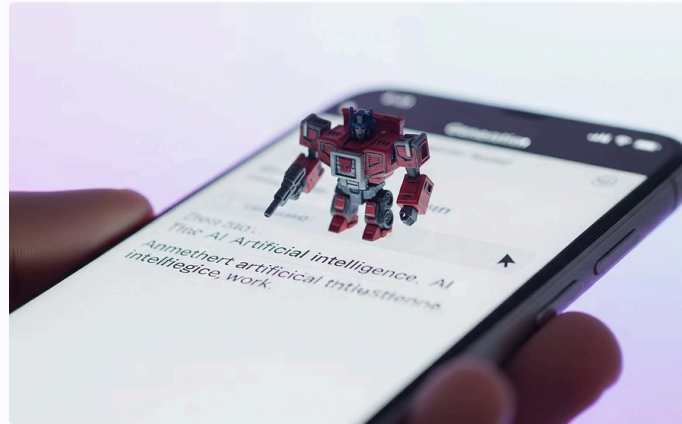


Real-World Edge Examples



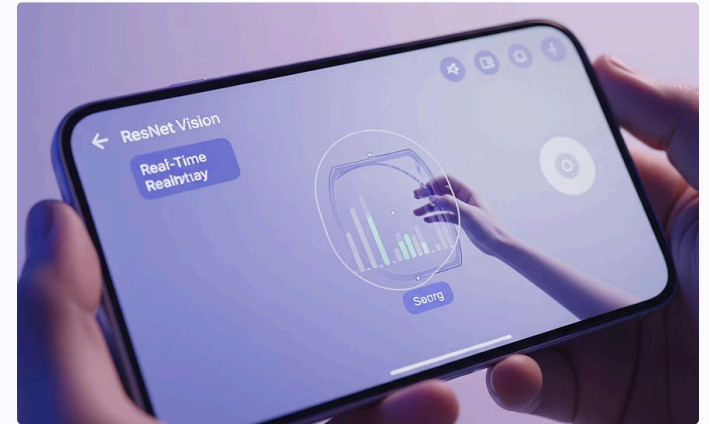
YOLOv5 on Jetson Nano

Pruned by 40% → runs 2× faster with only 2.5% mAP drop



Transformer on Mobile

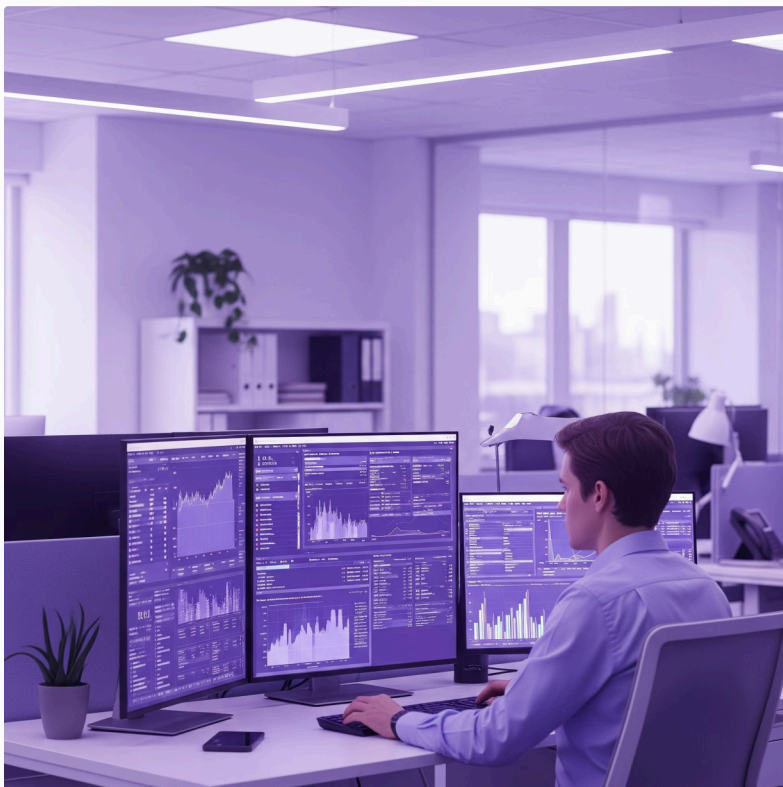
Heads/layers pruned by 30% → fits in 2GB RAM for on-device NLP



ResNet Vision Apps

Filter pruning + quantization → 4× smaller with < 1% accuracy drop

Best Practices



Start Conservative

Begin with **small pruning ratios** (10–30%)

Always Fine-tune

Fine-tune after pruning to recover accuracy

Combine Methods

Pair pruning + quantization for best gains

Favor Structure

Use **structured pruning** for hardware efficiency

Measure What Matters

Benchmark latency, not just parameter count

Key Takeaways



Pruning Fundamentals

Removing unnecessary weights/filters based on importance metrics



Triple Benefits

Reduces **model size, FLOPs, and power use**



Structure Matters

Structured pruning is most **edge-device friendly**



Combine Techniques

Works best when combined with **quantization & distillation**

Start your edge deployment journey with model compression today!

Contact: edge-ml-team@yourcompany.com | Resources: github.com/yourorg/pruning-toolkit